

7.3 Positional Lists

When working with array-based sequences, integer indices provide an excellent means for describing the location of an element, or the location at which an insertion or deletion should take place. However, numeric indices are not a good choice for describing positions within a linked list because, knowing only an element's index, the only way to reach it is to traverse the list incrementally from its beginning or end, counting elements along the way.

Furthermore, indices are not a good abstraction for describing a more local view of a position in a sequence, because the index of an entry changes over time due to insertions or deletions that happen earlier in the sequence. For example, it may not be convenient to describe the location of a person waiting in line based on the index, as that requires knowledge of precisely how far away that person is from the front of the line. We prefer an abstraction, as characterized in Figure 7.7, in which there is some other means for describing a position.

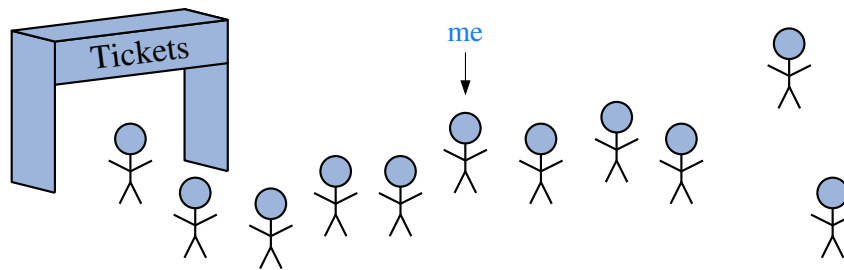


Figure 7.7: We wish to be able to identify the position of an element in a sequence without the use of an integer index. The label “me” represents some abstraction that identifies the position.

Our goal is to design an abstract data type that provides a user a way to refer to elements anywhere in a sequence, and to perform arbitrary insertions and deletions. This would allow us to efficiently describe actions such as a person deciding to leave the line before reaching the front, or allowing a friend to “cut” into line right behind him or her.

As another example, a text document can be viewed as a long sequence of characters. A word processor uses the abstraction of a **cursor** to describe a position within the document without explicit use of an integer index, allowing operations such as “delete the character at the cursor” or “insert a new character just after the cursor.” Furthermore, we may be able to refer to an inherent position within a document, such as the beginning of a particular chapter, without relying on a character index (or even a chapter number) that may change as the document evolves.

For these reasons, we temporarily forego the index-based methods of Java's formal List interface, and instead develop our own abstract data type that we denote as a **positional list**. Although a positional list is an abstraction, and need not rely on a linked list for its implementation, we certainly have a linked list in mind as we design the ADT, ensuring that it takes best advantage of particular capabilities of a linked list, such as $O(1)$ -time insertions and deletions at arbitrary positions (something that is not possible with an array-based sequence).

We face an immediate challenge in designing the ADT; to achieve constant time insertions and deletions at arbitrary locations, we effectively need a reference to the node at which an element is stored. It is therefore very tempting to develop an ADT in which a node reference serves as the mechanism for describing a position. In fact, our DoublyLinkedList class of Section 3.4.1 has methods addBetween and remove that accept node references as parameters; however, we intentionally declared those methods as private.

Unfortunately, the public use of nodes in the ADT would violate the object-oriented design principles of abstraction and encapsulation, which were introduced in Chapter 2. There are several reasons to prefer that we encapsulate the nodes of a linked list, for both our sake and for the benefit of users of our abstraction:

- It will be simpler for users of our data structure if they are not bothered with unnecessary details of our implementation, such as low-level manipulation of nodes, or our reliance on the use of sentinel nodes. Notice that to use the addBetween method of our DoublyLinkedList class to add a node at the beginning of a sequence, the header sentinel must be sent as a parameter.
- We can provide a more robust data structure if we do not permit users to directly access or manipulate the nodes. We can then ensure that users do not invalidate the consistency of a list by mismanaging the linking of nodes. A more subtle problem arises if a user were allowed to call the addBetween or remove method of our DoublyLinkedList class, sending a node that does not belong to the given list as a parameter. (Go back and look at that code and see why it causes a problem!)
- By better encapsulating the internal details of our implementation, we have greater flexibility to redesign the data structure and improve its performance. In fact, with a well-designed abstraction, we can provide a notion of a nonnumeric position, even if using an array-based sequence. (See Exercise C-7.43.)

Therefore, in defining the positional list ADT, we also introduce the concept of a **position**, which formalizes the intuitive notion of the “location” of an element relative to others in the list. (When we do use a linked list for the implementation, we will later see how we can privately use node references as natural manifestations of positions.)

7.3.1 Positions

To provide a general abstraction for the location of an element within a structure, we define a simple **position** abstract data type. A position supports the following single method:

`getElement()`: Returns the element stored at this position.

A position acts as a marker or token within a broader positional list. A position p , which is associated with some element e in a list L , does not change, even if the index of e changes in L due to insertions or deletions elsewhere in the list. Nor does position p change if we replace the element e stored at p with another element. The only way in which a position becomes invalid is if that position (and its element) are explicitly removed from the list.

Having a formal definition of a position type allows positions to serve as parameters to some methods and return values from other methods of the positional list ADT, which we next describe.

7.3.2 The Positional List Abstract Data Type

We now view a **positional list** as a collection of positions, each of which stores an element. The accessor methods provided by the positional list ADT include the following, for a list L :

`first()`: Returns the position of the first element of L (or null if empty).

`last()`: Returns the position of the last element of L (or null if empty).

`before( $p$ )`: Returns the position of L immediately before position p (or null if p is the first position).

`after( $p$ )`: Returns the position of L immediately after position p (or null if p is the last position).

`isEmpty()`: Returns true if list L does not contain any elements.

`size()`: Returns the number of elements in list L .

An error occurs if a position p , sent as a parameter to a method, is not a valid position for the list.

Note well that the `first()` and `last()` methods of the positional list ADT return the associated *positions*, not the *elements*. (This is in contrast to the corresponding `first` and `last` methods of the deque ADT.) The first element of a positional list can be determined by subsequently invoking the `getElement` method on that position, as `first().getElement`. The advantage of receiving a position as a return value is that we can subsequently use that position to traverse the list.

As a demonstration of a typical traversal of a positional list, Code Fragment 7.6 traverses a list, named `guests`, that stores string elements, and prints each element while traversing from the beginning of the list to the end.

```
1 Position<String> cursor = guests.first();
2 while (cursor != null) {
3     System.out.println(cursor.getElement());
4     cursor = guests.after(cursor);           // advance to the next position (if any)
5 }
```

Code Fragment 7.6: A traversal of a positional list.

This code relies on the convention that the null reference is returned when the `after` method is called upon the last position. (That return value is clearly distinguishable from any legitimate position.) The positional list ADT similarly indicates that the null value is returned when the `before` method is invoked at the front of the list, or when `first` or `last` methods are called upon an empty list. Therefore, the above code fragment works correctly even if the `guests` list is empty.

Updated Methods of a Positional List

The positional list ADT also includes the following *update* methods:

addFirst(*e*): Inserts a new element *e* at the front of the list, returning the position of the new element.

addLast(*e*): Inserts a new element *e* at the back of the list, returning the position of the new element.

addBefore(*p*, *e*): Inserts a new element *e* in the list, just before position *p*, returning the position of the new element.

addAfter(*p*, *e*): Inserts a new element *e* in the list, just after position *p*, returning the position of the new element.

set(*p*, *e*): Replaces the element at position *p* with element *e*, returning the element formerly at position *p*.

remove(*p*): Removes and returns the element at position *p* in the list, invalidating the position.

There may at first seem to be redundancy in the above repertoire of operations for the positional list ADT, since we can perform operation `addFirst(e)` with `addBefore(first(), e)`, and operation `addLast(e)` with `addAfter(last(), e)`. But these substitutions can only be done for a nonempty list.

Example 7.4: The following table shows a series of operations on an initially empty positional list storing integers. To identify position instances, we use variables such as p and q . For ease of exposition, when displaying the list contents, we use subscript notation to denote the position storing an element.

Method	Return Value	List Contents
addLast(8)	p	(8_p)
first()	p	(8_p)
addAfter(p , 5)	q	$(8_p, 5_q)$
before(q)	p	$(8_p, 5_q)$
addBefore(q , 3)	r	$(8_p, 3_r, 5_q)$
r .getElement()	3	$(8_p, 3_r, 5_q)$
after(p)	r	$(8_p, 3_r, 5_q)$
before(p)	null	$(8_p, 3_r, 5_q)$
addFirst(9)	s	$(9_s, 8_p, 3_r, 5_q)$
remove(last())	5	$(9_s, 8_p, 3_r)$
set(p , 7)	8	$(9_s, 7_p, 3_r)$
remove(q)	"error"	$(9_s, 7_p, 3_r)$

Java Interface Definitions

We are now ready to formalize the position ADT and positional list ADT. A Java Position interface, representing the position ADT, is given in Code Fragment 7.7. Following that, Code Fragment 7.8 presents a Java definition for our PositionalList interface. If the getElement() method is called on a Position instance that has previously been removed from its list, an IllegalStateException is thrown. If an invalid Position instance is sent as a parameter to a method of a PositionalList, an IllegalArgumentException is thrown. (Both of those exception types are defined in the standard Java hierarchy.)

```

1 public interface Position<E> {
2     /**
3      * Returns the element stored at this position.
4      *
5      * @return the stored element
6      * @throws IllegalStateException if position no longer valid
7      */
8     E getElement() throws IllegalStateException;
9 }

```

Code Fragment 7.7: The Position interface.

```
1  /** An interface for positional lists. */
2  public interface PositionalList<E> {
3
4      /** Returns the number of elements in the list. */
5      int size();
6
7      /** Tests whether the list is empty. */
8      boolean isEmpty();
9
10     /** Returns the first Position in the list (or null, if empty). */
11     Position<E> first();
12
13     /** Returns the last Position in the list (or null, if empty). */
14     Position<E> last();
15
16     /** Returns the Position immediately before Position p (or null, if p is first). */
17     Position<E> before(Position<E> p) throws IllegalArgumentException;
18
19     /** Returns the Position immediately after Position p (or null, if p is last). */
20     Position<E> after(Position<E> p) throws IllegalArgumentException;
21
22     /** Inserts element e at the front of the list and returns its new Position. */
23     Position<E> addFirst(E e);
24
25     /** Inserts element e at the back of the list and returns its new Position. */
26     Position<E> addLast(E e);
27
28     /** Inserts element e immediately before Position p and returns its new Position. */
29     Position<E> addBefore(Position<E> p, E e)
30         throws IllegalArgumentException;
31
32     /** Inserts element e immediately after Position p and returns its new Position. */
33     Position<E> addAfter(Position<E> p, E e)
34         throws IllegalArgumentException;
35
36     /** Replaces the element stored at Position p and returns the replaced element. */
37     E set(Position<E> p, E e) throws IllegalArgumentException;
38
39     /** Removes the element stored at Position p and returns it (invalidating p). */
40     E remove(Position<E> p) throws IllegalArgumentException;
41 }
```

Code Fragment 7.8: The PositionalList interface.

7.3.3 Doubly Linked List Implementation

Not surprisingly, our preferred implementation of the `PositionalList` interface relies on a doubly linked list. Although we implemented a `DoublyLinkedList` class in Chapter 3, that class does not adhere to the `PositionalList` interface.

In this section, we develop a concrete implementation of the `PositionalList` interface using a doubly linked list. The low-level details of our new linked-list representation, such as the use of header and trailer sentinels, will be identical to our earlier version; we refer the reader to Section 3.4 for a discussion of the doubly linked list operations. What differs in this section is our management of the positional abstraction.

The obvious way to identify locations within a linked list are node references. Therefore, we declare the nested `Node` class of our linked list so as to implement the `Position` interface, supporting the required `getElement` method. So the nodes *are* the positions. Yet, the `Node` class is declared as private, to maintain proper encapsulation. All of the public methods of the positional list rely on the `Position` type, so although we know we are sending and receiving nodes, these are only known to be positions from the outside; as a result, users of our class cannot call any method other than `getElement()`.

In Code Fragments 7.9–7.12, we define a `LinkedPositionalList` class, which implements the positional list ADT. We provide the following guide to that code:

- Code Fragment 7.9 contains the definition of the nested `Node<E>` class, which implements the `Position<E>` interface. Following that are the declaration of the instance variables of the outer `LinkedPositionalList` class and its constructor.
- Code Fragment 7.10 begins with two important utility methods that help us robustly cast between the `Position` and `Node` types. The `validate(p)` method is called anytime the user sends a `Position` instance as a parameter. It throws an exception if it determines that the position is invalid, and otherwise returns that instance, implicitly cast as a `Node`, so that methods of the `Node` class can subsequently be called. The private `position(node)` method is used when about to return a `Position` to the user. Its primary purpose is to make sure that we do not expose either sentinel node to a caller, returning a **null** reference in such a case. We rely on both of these private utility methods in the public accessor methods that follow.
- Code Fragment 7.11 provides most of the public update methods, relying on a private `addBetween` method to unify the implementations of the various insertion operations.
- Code Fragment 7.12 provides the public `remove` method. Note that it sets all fields of the removed node back to null—a condition we can later detect to recognize a defunct position.

```

1  /** Implementation of a positional list stored as a doubly linked list. */
2  public class LinkedPositionalList<E> implements PositionalList<E> {
3      //----- nested Node class -----
4      private static class Node<E> implements Position<E> {
5          private E element;           // reference to the element stored at this node
6          private Node<E> prev;        // reference to the previous node in the list
7          private Node<E> next;        // reference to the subsequent node in the list
8          public Node(E e, Node<E> p, Node<E> n) {
9              element = e;
10             prev = p;
11             next = n;
12         }
13         public E getElement() throws IllegalStateException {
14             if (next == null)           // convention for defunct node
15                 throw new IllegalStateException("Position no longer valid");
16             return element;
17         }
18         public Node<E> getPrev() {
19             return prev;
20         }
21         public Node<E> getNext() {
22             return next;
23         }
24         public void setElement(E e) {
25             element = e;
26         }
27         public void setPrev(Node<E> p) {
28             prev = p;
29         }
30         public void setNext(Node<E> n) {
31             next = n;
32         }
33     } //----- end of nested Node class -----
34
35     // instance variables of the LinkedPositionalList
36     private Node<E> header;           // header sentinel
37     private Node<E> trailer;          // trailer sentinel
38     private int size = 0;              // number of elements in the list
39
40     /** Constructs a new empty list. */
41     public LinkedPositionalList() {
42         header = new Node<>(null, null, null); // create header
43         trailer = new Node<>(null, header, null); // trailer is preceded by header
44         header.setNext(trailer);           // header is followed by trailer
45     }

```

Code Fragment 7.9: An implementation of the `LinkedPositionalList` class.
(Continues in Code Fragments 7.10–7.12.)


```

46 // private utilities
47 /** Validates the position and returns it as a node. */
48 private Node<E> validate(Position<E> p) throws IllegalArgumentException {
49     if (!(p instanceof Node)) throw new IllegalArgumentException("Invalid p");
50     Node<E> node = (Node<E>) p; // safe cast
51     if (node.getNext() == null) // convention for defunct node
52         throw new IllegalArgumentException("p is no longer in the list");
53     return node;
54 }
55
56 /** Returns the given node as a Position (or null, if it is a sentinel). */
57 private Position<E> position(Node<E> node) {
58     if (node == header || node == trailer)
59         return null; // do not expose user to the sentinels
60     return node;
61 }
62
63 // public accessor methods
64 /** Returns the number of elements in the linked list. */
65 public int size() { return size; }
66
67 /** Tests whether the linked list is empty. */
68 public boolean isEmpty() { return size == 0; }
69
70 /** Returns the first Position in the linked list (or null, if empty). */
71 public Position<E> first() {
72     return position(header.getNext());
73 }
74
75 /** Returns the last Position in the linked list (or null, if empty). */
76 public Position<E> last() {
77     return position(trailer.getPrev());
78 }
79
80 /** Returns the Position immediately before Position p (or null, if p is first). */
81 public Position<E> before(Position<E> p) throws IllegalArgumentException {
82     Node<E> node = validate(p);
83     return position(node.getPrev());
84 }
85
86 /** Returns the Position immediately after Position p (or null, if p is last). */
87 public Position<E> after(Position<E> p) throws IllegalArgumentException {
88     Node<E> node = validate(p);
89     return position(node.getNext());
90 }

```

Code Fragment 7.10: An implementation of the `LinkedPositionalList` class.
(Continued from Code Fragment 7.9; continues in Code Fragments 7.11 and 7.12.)

```

91 // private utilities
92 /** Adds element e to the linked list between the given nodes. */
93 private Position<E> addBetween(E e, Node<E> pred, Node<E> succ) {
94     Node<E> newest = new Node<>(e, pred, succ); // create and link a new node
95     pred.setNext(newest);
96     succ.setPrev(newest);
97     size++;
98     return newest;
99 }
100
101 // public update methods
102 /** Inserts element e at the front of the linked list and returns its new Position. */
103 public Position<E> addFirst(E e) {
104     return addBetween(e, header, header.getNext()); // just after the header
105 }
106
107 /** Inserts element e at the back of the linked list and returns its new Position. */
108 public Position<E> addLast(E e) {
109     return addBetween(e, trailer.getPrev(), trailer); // just before the trailer
110 }
111
112 /** Inserts element e immediately before Position p, and returns its new Position.*/
113 public Position<E> addBefore(Position<E> p, E e)
114     throws IllegalArgumentException {
115     Node<E> node = validate(p);
116     return addBetween(e, node.getPrev(), node);
117 }
118
119 /** Inserts element e immediately after Position p, and returns its new Position. */
120 public Position<E> addAfter(Position<E> p, E e)
121     throws IllegalArgumentException {
122     Node<E> node = validate(p);
123     return addBetween(e, node, node.getNext());
124 }
125
126 /** Replaces the element stored at Position p and returns the replaced element. */
127 public E set(Position<E> p, E e) throws IllegalArgumentException {
128     Node<E> node = validate(p);
129     E answer = node.getElement();
130     node.setElement(e);
131     return answer;
132 }

```

Code Fragment 7.11: An implementation of the `LinkedPositionalList` class.
 (Continued from Code Fragments 7.9 and 7.10; continues in Code Fragment 7.12.)

```

133  /** Removes the element stored at Position p and returns it (invalidating p). */
134  public E remove(Position<E> p) throws IllegalArgumentException {
135      Node<E> node = validate(p);
136      Node<E> predecessor = node.getPrev();
137      Node<E> successor = node.getNext();
138      predecessor.setNext(successor);
139      successor.setPrev(predecessor);
140      size--;
141      E answer = node.getElement();
142      node.setElement(null);           // help with garbage collection
143      node.setNext(null);             // and convention for defunct node
144      node.setPrev(null);
145      return answer;
146  }
147  }

```

Code Fragment 7.12: An implementation of the `LinkedPositionalList` class.
(Continued from Code Fragments 7.9–7.11.)

The Performance of a Linked Positional List

The positional list ADT is ideally suited for implementation with a doubly linked list, as all operations run in worst-case constant time, as shown in Table 7.2. This is in stark contrast to the `ArrayList` structure (analyzed in Table 7.1), which requires linear time for insertions or deletions at arbitrary positions, due to the need for a loop to shift other elements.

Of course, our positional list does not support the index-based methods of the official `List` interface of Section 7.1. It is possible to add support for those methods by traversing the list while counting nodes (see Exercise C-7.38), but that requires time proportional to the sublist that is traversed.

Method	Running Time
<code>size()</code>	$O(1)$
<code>isEmpty()</code>	$O(1)$
<code>first()</code> , <code>last()</code>	$O(1)$
<code>before(p)</code> , <code>after(p)</code>	$O(1)$
<code>addFirst(e)</code> , <code>addLast(e)</code>	$O(1)$
<code>addBefore(p, e)</code> , <code>addAfter(p, e)</code>	$O(1)$
<code>set(p, e)</code>	$O(1)$
<code>remove(p)</code>	$O(1)$

Table 7.2: Performance of a positional list with n elements realized by a doubly linked list. The space usage is $O(n)$.

Implementing a Positional List with an Array

We can implement a positional list L using an array A for storage, but some care is necessary in designing objects that will serve as positions. At first glance, it would seem that a position p need only store the index i at which its associated element is stored within the array. We can then implement method `getElement( $p$ )` simply by returning $A[i]$. The problem with this approach is that the index of an element e changes when other insertions or deletions occur before it. If we have already returned a position p associated with element e that stores an outdated index i to a user, the wrong array cell would be accessed when the position was used. (Remember that positions in a positional list should always be defined relative to their neighboring positions, not their indices.)

Hence, if we are going to implement a positional list with an array, we need a different approach. We recommend the following representation. Instead of storing the elements of L directly in array A , we store a new kind of position object in each cell of A . A position p stores the element e as well as the current index i of that element within the list. Such a data structure is illustrated in Figure 7.8.

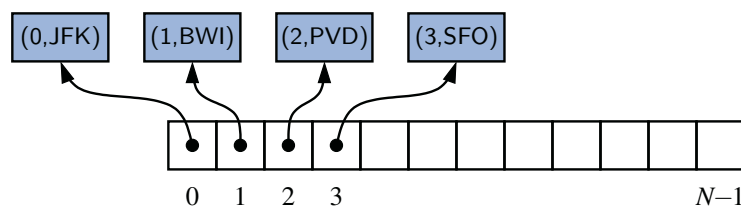


Figure 7.8: An array-based representation of a positional list.

With this representation, we can determine the index currently associated with a position, and we can determine the position currently associated with a specific index. We can therefore implement an accessor, such as `before( $p$ )`, by finding the index of the given position and using the array to find the neighboring position.

When an element is inserted or deleted somewhere in the list, we can loop through the array to update the index variable stored in all later positions in the list that are shifted during the update.

Efficiency Trade-Offs with an Array-Based Sequence

In this array implementation of a sequence, the `addFirst`, `addBefore`, `addAfter`, and `remove` methods take $O(n)$ time, because we have to shift position objects to make room for the new position or to fill in the hole created by the removal of the old position (just as in the insert and remove methods based on index). All the other position-based methods take $O(1)$ time.